

DAY 1

Machine Learning

Supervised Learning + Regression + ML Pipeline

Complete Study Notes

↪ DAY 1 GOAL

Understand the end-to-end Machine Learning workflow and build regression models.

By the end of this day you will:

- Understand the AI/ML/DL/GenAI hierarchy
- Know each stage of the ML pipeline
- Grasp key data concepts (features, labels, splits)
- Build Simple and Multiple Linear Regression models
- Know when regression works — and when it fails

SECTION 1: AI vs ML vs DL vs Generative AI

Understanding the hierarchy — Quick but Clear

1.1 The Umbrella Hierarchy

These four terms are often confused. Think of them as nested circles — each one lives inside the previous.

AI (Outermost)	Machine Learning	Deep Learning	Generative AI
Artificial Intelligence	Subset of AI	Subset of ML	Subset / Application of DL
Making machines mimic human intelligence	Learning patterns from data	Learning using layered neural networks	Creating new content (text, images, code)
Rule-based + learning systems	Algorithms: Linear Reg, SVM, Trees	CNNs, RNNs, Transformers	GPT, DALL-E, Stable Diffusion
Chess engine, spam filter, Siri	House price prediction, churn model	Image recognition, speech-to-text	ChatGPT, Claude, Midjourney

1.2 Deep Dive: Each Layer Explained

Artificial Intelligence (AI)

The broadest umbrella. AI is any technique that enables machines to simulate human-like intelligence — whether through hard-coded rules or learned behaviour. AI existed before machine learning (e.g. expert systems in the 1980s).

Machine Learning (ML)

Instead of manually writing rules, the machine learns patterns from data. The key idea: give examples → let the algorithm find the rules itself.

Key ML Insight

Traditional programming: Data + Rules → Answers

Machine Learning: Data + Answers → Rules (Model)

Deep Learning (DL)

Deep Learning uses multi-layered Artificial Neural Networks (ANNs) inspired by the human brain. "Deep" refers to many hidden layers. DL automatically extracts features — no manual feature engineering needed for images or audio.

Generative AI (GenAI)

The newest and most hyped layer. GenAI models are trained on massive datasets to generate new, realistic content — text, images, code, video, audio.

1.3 Where do Modern LLMs Fit?

LLM Placement in the Hierarchy

LLMs (Large Language Models) such as GPT-4, Claude, Gemini sit at the intersection of:

- Deep Learning — built on Transformer neural network architecture
- Generative AI — generate new text sequences token by token
- Machine Learning — trained by learning from data (next-token prediction / RLHF)

Path: AI $\supset$ ML $\supset$ DL $\supset$ GenAI $\supset$ LLM

LLMs are pre-trained on internet-scale text data and fine-tuned with human feedback (RLHF). They understand context, reason over text, write code, and generate human-like responses.

SECTION 2: Machine Learning Pipeline

Core Foundation — Structure + Reproducibility

2.1 Why a Pipeline?

A pipeline is a structured sequence of steps to go from raw data to a deployed model. Just like a production line in manufacturing, each stage has a specific job, and skipping or scrambling steps causes failure.

Real-World Analogy: Cooking Pipeline

1. Gather ingredients → Data Collection
2. Wash & chop → Data Preprocessing
3. Season & mix → Feature Engineering
4. Cook on stove → Model Training
5. Taste & adjust → Evaluation
6. Serve to customer → Deployment / Prediction

Skipping step 2 (washing ingredients) ruins the final dish — just like dirty data ruins ML models.

2.2 The 5 Stages of the ML Pipeline

Stage 1: Data Collection

The foundation. Garbage in = garbage out. Collect relevant, sufficient, and representative data.

- Sources: CSV files, databases, APIs, web scraping, sensors, surveys
- More data generally = better model (up to a point)
- Data quality matters more than quantity

Stage 2: Data Preprocessing

Raw data is almost always messy. Preprocessing cleans and prepares it for algorithms.

- Handle missing values (fill or drop)
- Remove duplicates and fix errors
- Encode categorical variables (text → numbers)
- Scale/normalize numerical features

Stage 3: Feature Engineering

Transform raw variables into features that better represent the underlying patterns. This is where domain knowledge matters most.

- Create new features from existing ones (e.g. age from birthdate)
- Select relevant features, remove irrelevant ones
- Reduce dimensionality if needed (PCA)

Stage 4: Model Training

Feed the prepared data into a chosen algorithm and let it learn patterns (parameters/weights).

- Choose algorithm based on problem type (regression, classification, clustering...)
- Fit the model on training data
- Tune hyperparameters for better performance

Stage 5: Evaluation

Measure how well the model performs on unseen data. Never evaluate on training data alone.

- Use metrics: MSE, RMSE, MAE, R^2 (for regression)
- Compare against a baseline
- Check for overfitting or underfitting

2.3 Pipeline Flow Diagram



← Each arrow represents: output of one stage becomes input of the next →

SECTION 3: Data Concepts

Features, Labels, Structured vs Unstructured, Train vs Test

3.1 Features vs Labels

Features (X) — Inputs	Labels (y) — Output
Independent variables	Dependent variable (what you predict)
Also called: predictors, attributes, columns	Also called: target, response, output
e.g. House size, bedrooms, location	e.g. House price
e.g. Age, salary, credit score	e.g. Loan approved (Yes/No)
Can be many columns	Usually one column (can be multiple)

3.2 Structured vs Unstructured Data

Structured Data	Unstructured Data
Organized in rows and columns	No fixed format
Spreadsheets, SQL databases, CSV files	Images, audio, video, text documents
Easily processed by traditional ML	Requires DL or NLP techniques
House prices, sales records, HR data	Customer reviews, X-rays, social media posts
Each column has a clear meaning	Meaning must be extracted algorithmically

3.3 Training Data vs Testing Data

Training Data	Testing Data
Model learns from this data	Model is evaluated on this data
Model sees labels during training	Labels are hidden — model must predict
Typically 70–80% of dataset	Typically 20–30% of dataset
More = better learning	Must remain completely unseen during training

3.4 Why Do We Split Data?

Critical Concept: Generalization

The whole point of ML is to make predictions on NEW, UNSEEN data.

If we train and evaluate on the SAME data:

- The model memorizes the training data (overfitting)
- It performs perfectly on training data, but fails on real-world data
- We get a falsely optimistic accuracy measurement

By keeping test data completely separate:

- We simulate real-world prediction scenarios
- We get an honest estimate of how the model will perform in production
- We can detect overfitting before deployment

SECTION 4: Train-Test Split

VERY IMPORTANT — Generalization & Best Practices

4.1 The 70/30 Split — Standard Starting Point

TRAINING SET (70%)

Model learns from this

TEST SET (30%)

Evaluate generalization

4.2 Why 70/30? — The Reasoning

- Training needs enough data to find meaningful patterns
- Testing needs enough data to give a reliable performance estimate
- 70/30 is a practical balance — not a law of nature
- The ratio is NOT fixed — it depends on dataset size and problem type

4.3 When to Use Different Splits

Split	Use When	Training Size	Notes
70/30	General purpose, medium datasets	70%	Standard starting point
80/20	Smaller datasets, need more training data	80%	Most common in practice
90/10	Very small datasets	90%	Test set may be unreliable
60/20/20	With validation set	60%	Train / Validation / Test

4.4 Cross-Validation (Brief Introduction)

What is Cross-Validation?

When dataset is small, a single train-test split may be misleading.

K-Fold Cross-Validation splits data into K equal parts (folds):

- Train on K-1 folds, test on 1 fold
- Repeat K times, each fold acts as test set once
- Average the K evaluation scores

Example: 5-Fold CV on 1000 samples:

Round 1: Train on rows 201-1000, Test on rows 1-200

Round 2: Train on rows 1-200 + 401-1000, Test on rows 201-400

... and so on

Result: More reliable estimate of model performance — especially for small datasets.

SECTION 5: Data Preprocessing

Refresher + Depth — Clean Data = Better Model

5.1 Handling Missing Values

Strategy	Method	When to Use	Risk
Drop rows	<code>df.dropna()</code>	Missing < 5% of data	Data loss
Fill with mean	<code>df.fillna(df.mean())</code>	Numerical, roughly normal	Distorts distribution
Fill with median	<code>df.fillna(df.median())</code>	Numerical with outliers	Safer than mean
Fill with mode	<code>df.fillna(df.mode()[0])</code>	Categorical data	May over-represent mode
Forward fill	<code>df.fillna(method='ffill')</code>	Time series data	Assumes continuity

5.2 Encoding Categorical Variables

Machine learning algorithms work with numbers. Categorical variables (text labels) must be converted.

Label Encoding	One-Hot Encoding
Assigns integer to each category: Red=0, Green=1, Blue=2	Creates binary column for each category
Implies an order: Blue > Green > Red (WRONG if no order!)	No false ordering implied
Use only for ordinal data (Low, Medium, High)	Use for nominal data (colours, cities, names)
sklearn: <code>LabelEncoder()</code>	pandas: <code>pd.get_dummies()</code> or <code>OneHotEncoder</code>
1 column output	N columns output (N = number of categories)

5.3 Feature Scaling — Intuition

Many algorithms are sensitive to the scale of features. If one feature is in millions (house price) and another is 0–1 (ratio), the large-scale feature will dominate learning.

Normalization (Min-Max Scaling)	Standardization (Z-Score)
Scales values to [0, 1] range	Scales to mean=0, std=1
Formula: $(x - \min) / (\max - \min)$	Formula: $(x - \text{mean}) / \text{std}$
Use when you need bounded output	Use when data is roughly normal

Affected by outliers strongly	Less affected by outliers
Good for: KNN, Neural Networks	Good for: Linear/Logistic Regression, SVM

5.4 Critical Rule: Avoid Data Leakage

Data Leakage Warning

Data leakage = information from the test set accidentally influences model training.

WRONG approach (causes leakage):

1. Scale all data → then split into train/test
(The scaler has seen test data statistics — leakage!)

CORRECT approach:

1. Split data into train/test FIRST
2. Fit scaler/encoder on TRAINING data ONLY
3. Transform BOTH train and test with the same fitted scaler

Rule: Test data should be treated as if it does not exist during preprocessing.

SECTION 6: Supervised Learning

Regression vs Classification

6.1 What is Supervised Learning?

In supervised learning, the model is trained on labeled data — every training example has both input features (X) and a known output label (y). The model learns to map $X \rightarrow y$.

Supervised Learning Formula

Training: Model learns $f(X) \approx y$ from (X, y) pairs

Prediction: Given new X^* , predict $y^* = f(X^*)$

The 'supervision' = the labels (correct answers) guide the learning process.

6.2 Regression vs Classification

Regression	Classification
Predicts a continuous numerical value	Predicts a discrete category/class
Output: any real number	Output: class label (e.g. 0, 1, 2, 'cat', 'dog')
Example: House price prediction	Example: Email spam detection
Example: Temperature forecast	Example: Tumour malignant or benign
Example: Stock price tomorrow	Example: Customer will churn or not
Metrics: MSE, RMSE, MAE, R^2	Metrics: Accuracy, Precision, Recall, F1
Algorithms: Linear Regression, Ridge, SVR	Algorithms: Logistic Regression, SVM, Trees

SECTION 7: Simple Linear Regression

One Feature → One Output

7.1 Core Concept

Simple Linear Regression fits a straight line through data points to model the relationship between ONE input feature (X) and ONE output (y).

The Line Equation

$$y = mx + b$$

Where:

y = predicted output (dependent variable)

x = input feature (independent variable)

m = slope (coefficient) — how much y changes per unit change in x

b = intercept — value of y when x = 0

In ML notation: $y = \beta_0 + \beta_1 x$

β_0 = intercept (bias)

β_1 = coefficient (weight)

7.2 How the Model Learns: Least Squares

The algorithm finds the values of β_0 and β_1 that minimize the total prediction error (residuals) across all training points.

Ordinary Least Squares (OLS) — The Math Intuition

For each data point i: $\text{error} = \text{actual_y}(i) - \text{predicted_y}(i)$

We minimize: $\text{MSE} = (1/n) \times \sum (\text{y_actual} - \text{y_predicted})^2$

Squaring the errors:

- Removes negative signs (penalizes over- and under-predictions equally)
- Penalizes large errors more than small ones
- Makes the math differentiable (we can find the minimum analytically)

The algorithm solves this in one step — no iteration needed (unlike gradient descent).

7.3 Prediction Mechanism

Once the line is fitted, prediction is trivial: plug in the new x value into the equation.

Prediction Example

Trained model: House Price = 50,000 + 200 × (Square Feet)

Predict price for a 1,500 sq ft house:

$$\text{Price} = 50,000 + 200 \times 1,500$$

$$\text{Price} = 50,000 + 300,000$$

$$\text{Price} = \$350,000$$

7.4 When to Use Simple Linear Regression

USE WHEN	DON'T USE WHEN	BEST PRACTICES
Relationship is linear Interpretability matters Quick baseline needed One main predictor Small to medium dataset	Data is non-linear (curves) Complex interaction patterns Many features needed Data has many outliers Image / text / audio tasks	Plot X vs y first (scatter) Check linearity assumption Remove extreme outliers Check residual plot Scale features if needed

SECTION 8: Multiple Linear Regression

Multiple Features → One Output

8.1 Core Concept

Most real-world problems depend on MORE than one factor. Multiple Linear Regression extends the concept to handle multiple input features simultaneously.

The Multi-Feature Equation

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n$$

Example: House Price Prediction

$$\text{Price} = 20,000 + 150 \times (\text{SqFt}) + 10,000 \times (\text{Bedrooms}) - 500 \times (\text{Age}) + 5,000 \times (\text{Bathrooms})$$

Each β_i (coefficient) tells us:

- How much y changes for a 1-unit increase in x_i
- HOLDING ALL OTHER FEATURES CONSTANT (ceteris paribus)

8.2 Why Use Multiple Features?

A house price depends on size, location, age, number of rooms, amenities — not just one thing. Real world problems are multidimensional. More relevant features = better model.

Multiple vs Simple — Key Difference

Simple: Price = $f(\text{SqFt only})$ — ignores all other factors

Multiple: Price = $f(\text{SqFt, Bedrooms, Age, Location, ...})$ — realistic

Multiple regression captures the combined effect of all features together.

This is usually significantly more accurate in practice.

8.3 When to Use Multiple Linear Regression

USE WHEN	AVOID WHEN
Multiple relevant features exist Linear relationship between each X and y Features are relatively independent	Too many irrelevant features (noise) Strong non-linear relationships exist High multicollinearity (correlated features)

Interpretability of each feature's impact needed

More features than data rows

8.4 Multicollinearity — Critical Warning

What is Multicollinearity?

Multicollinearity occurs when two or more features are highly correlated with each other.

Example: Including both 'Height in cm' and 'Height in inches' — same information!

Example: Including both 'Salary' and 'Monthly income' — linearly dependent!

Why it is a problem:

- Coefficients become unstable and unreliable
- Small changes in data cause huge coefficient swings
- Hard to interpret which feature 'really' matters

How to detect: Correlation matrix, Variance Inflation Factor (VIF)

How to fix: Remove correlated features, use PCA, or apply Ridge regression

SECTION 9: Model Evaluation

Metrics, Overfitting, Underfitting

9.1 Why Evaluate?

Building a model is just step one. We must measure HOW WELL it performs — and catch problems like overfitting before deploying it to production.

9.2 Regression Evaluation Metrics

Metric	Formula	Interpretation	Ideal Value
MAE (Mean Absolute Error)	Mean of actual - predicted	Average absolute error — easy to interpret	Close to 0
MSE (Mean Squared Error)	Mean of (actual - predicted) ²	Penalizes large errors more — sensitive to outliers	Close to 0
RMSE (Root MSE)	$\sqrt{\text{MSE}}$	Same units as y — most commonly reported	Close to 0
R ² (R-Squared)	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$	% of variance explained by model	Close to 1 (max = 1)

9.3 R² — Deep Understanding

Interpreting R-Squared

R² = 0.85 means: 'The model explains 85% of the variance in the target variable'

R² = 1.0 → Perfect fit (model predicts every point exactly — likely overfitting!)

R² = 0.8 → Good: model captures most patterns

R² = 0.5 → Moderate: room for improvement

R² = 0.0 → Model does no better than just predicting the mean

R² < 0 → Model is worse than predicting the mean (something is wrong!)

WARNING: High R² on training data means nothing. Always check R² on test data.

9.4 Overfitting vs Underfitting

Overfitting	Good Fit	Underfitting
Model memorizes training data Performs perfectly on train Fails on test data Model too complex <i>Fix: Simplify model, get more data, regularization</i>	Captures true patterns Good train AND test performance Generalizes well to new data Appropriate model complexity This is the goal!	Model too simple Misses important patterns Poor on both train and test High bias <i>Fix: More features, complex model</i>

SECTION 10: Hands-On Code

Full ML Pipeline in Python

10.1 Complete Pipeline: Simple Linear Regression

```
# -----
# STEP 1: Import Libraries
# -----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# -----
# STEP 2: Load Dataset
# -----
df = pd.read_csv('housing.csv')
print(df.head())
print(df.info())      # Check dtypes and nulls
print(df.describe())  # Statistical summary

# -----
# STEP 3: Preprocessing
# -----
# Drop rows with missing values
df = df.dropna()

# Select feature and label
X = df[['square_feet']] # Single feature (2D array needed)
y = df['price']          # Target variable

# -----
# STEP 4: Train-Test Split (70/30)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

print(f'Training samples: {len(X_train)}')
print(f'Testing samples: {len(X_test)}')

# -----
# STEP 5: Feature Scaling (fit on train only!)
# -----
scaler = StandardScaler()
```

```

X_train_scaled = scaler.fit_transform(X_train) # Fit + Transform
X_test_scaled = scaler.transform(X_test)      # Transform ONLY

# -----
# STEP 6: Train Simple Linear Regression
# -----
model = LinearRegression()
model.fit(X_train_scaled, y_train)

print(f'Coefficient (slope):      {model.coef_[0]:.2f}')
print(f'Intercept (bias):        {model.intercept_:.2f}')

# -----
# STEP 7: Make Predictions
# -----
y_pred = model.predict(X_test_scaled)

# -----
# STEP 8: Evaluate
# -----
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f'MAE: {mae:.2f}')
print(f'MSE: {mse:.2f}')
print(f'RMSE: {rmse:.2f}')
print(f'R²: {r2:.4f}')

```

10.2 Multiple Linear Regression Extension

```

# -----
# Multiple Linear Regression – use multiple features
# -----

# Select multiple features
feature_cols = ['square_feet', 'bedrooms', 'bathrooms', 'age']
X_multi = df[feature_cols]
y = df['price']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X_multi, y, test_size=0.3, random_state=42)

# Scale
scaler = StandardScaler()

```

```
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# Train
multi_model = LinearRegression()
multi_model.fit(X_train_sc, y_train)

# Coefficients - how much each feature contributes
for col, coef in zip(feature_cols, multi_model.coef_):
    print(f'{col:15s}: {coef:.2f}')

# Predict & Evaluate
y_pred_multi = multi_model.predict(X_test_sc)
print(f'Multiple LR R2: {r2_score(y_test, y_pred_multi):.4f}')

# Check for multicollinearity using correlation matrix
corr_matrix = df[feature_cols].corr()
print(corr_matrix.round(2))
```

SECTION 11: Day 1 Summary & Quick Reference Cheatsheet

Everything in one place

Key Concepts Mastered Today

AI / ML / DL / GenAI Hierarchy

AI (umbrella) $\supset$ ML (learning from data) $\supset$ DL (neural networks) $\supset$ GenAI (content generation)

LLMs = DL + GenAI + Transformer architecture + trained by ML (RLHF)

ML Pipeline (5 Stages)

1. Data Collection $\rightarrow$ 2. Preprocessing $\rightarrow$ 3. Feature Engineering

4. Model Training $\rightarrow$ 5. Evaluation $\rightarrow$ (6. Deployment)

Never skip stages — pipeline order matters critically!

Data Splits

70/30 = General purpose split | 80/20 = When data is limited

Always: fit scaler on TRAIN only, transform both train and test (avoid leakage)

Cross-validation = K-Fold for small datasets

Linear Regression At a Glance

Simple LR: $y = \beta_0 + \beta_1 x$ (one feature)

Multiple LR: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$ (many features)

Training: Minimize MSE using Ordinary Least Squares

Evaluation: MAE / RMSE (error) + R^2 (variance explained)

Overfitting vs Underfitting — Diagnosing

Overfitting: Train $R^2 \gg$ Test $R^2 \rightarrow$ Model too complex

Underfitting: Low R^2 on both train & test $\rightarrow$ Model too simple

Good Fit: Train $\approx$ Test R^2 , both acceptably high

Algorithms & When to Use

Algorithm	Output Type	Key Assumption	Best Used For
Simple Linear Regression	Continuous number	Linear relationship	Single feature, interpretable result
Multiple Linear Regression	Continuous number	Linear, no multicollinearity	Multiple features, real-world datasets
Ridge Regression	Continuous number	Linear + L2 regularization	Many features, prevent overfitting
Lasso Regression	Continuous number	Linear + L1 regularization	Feature selection needed

□ End of Day 1

You have completed Day 1 of Machine Learning!

You can now:

- Explain AI vs ML vs DL vs Generative AI
- Describe each stage of the ML pipeline
- Understand features, labels, structured vs unstructured data
- Perform a train-test split and explain why
- Apply preprocessing: missing values, encoding, scaling
- Build Simple and Multiple Linear Regression models
- Evaluate models with MAE, RMSE, and R^2
- Identify overfitting and underfitting
- Know when Linear Regression works — and when to avoid it

Next Up: Day 2 — Classification, Logistic Regression & Decision Trees